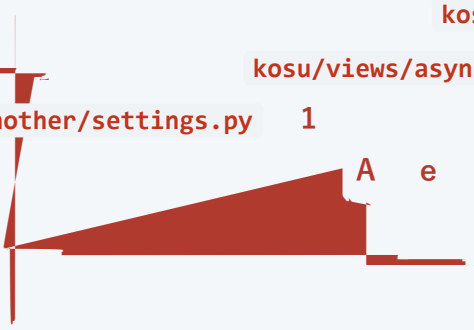
9 :

`kosu/tasks.py` 1

1 `kosu/views/asynchronous_views.py` 1

`hozen_another/settings.py` 1

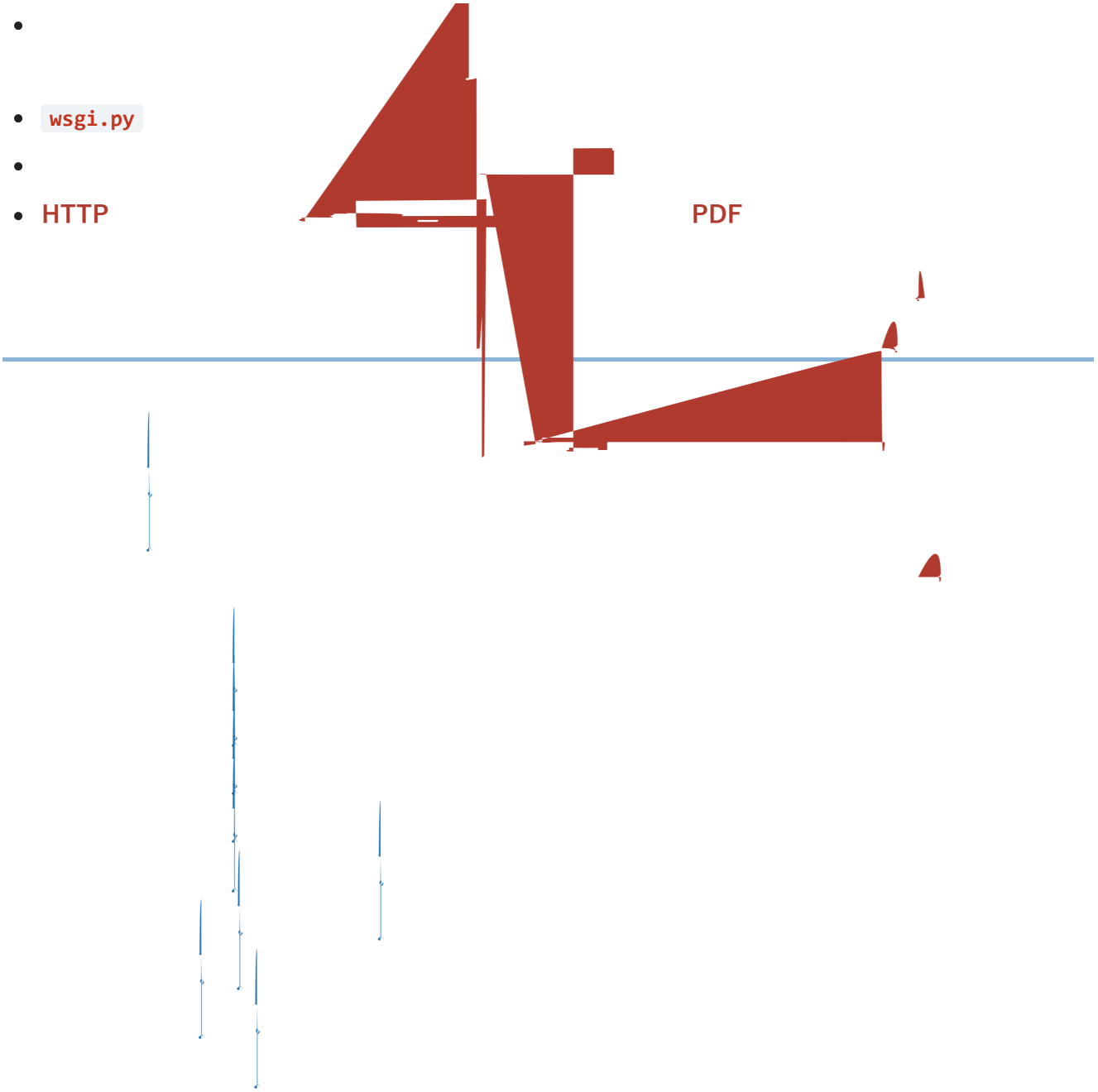
`wsgi.py`



- 3
-
-

- `tasks.py` 1 `openpyxl` `pandas`
- `asynchronous_views.py` 1
- `settings.py` 1 `LOGGING` `Q_CLUSTER` `DATABASES` `STATIC` `ALLOWED_HOSTS` `CSRF_TRUSTED_ORIGINS`

-
- `wsgi.py`
-
- HTTP PDF



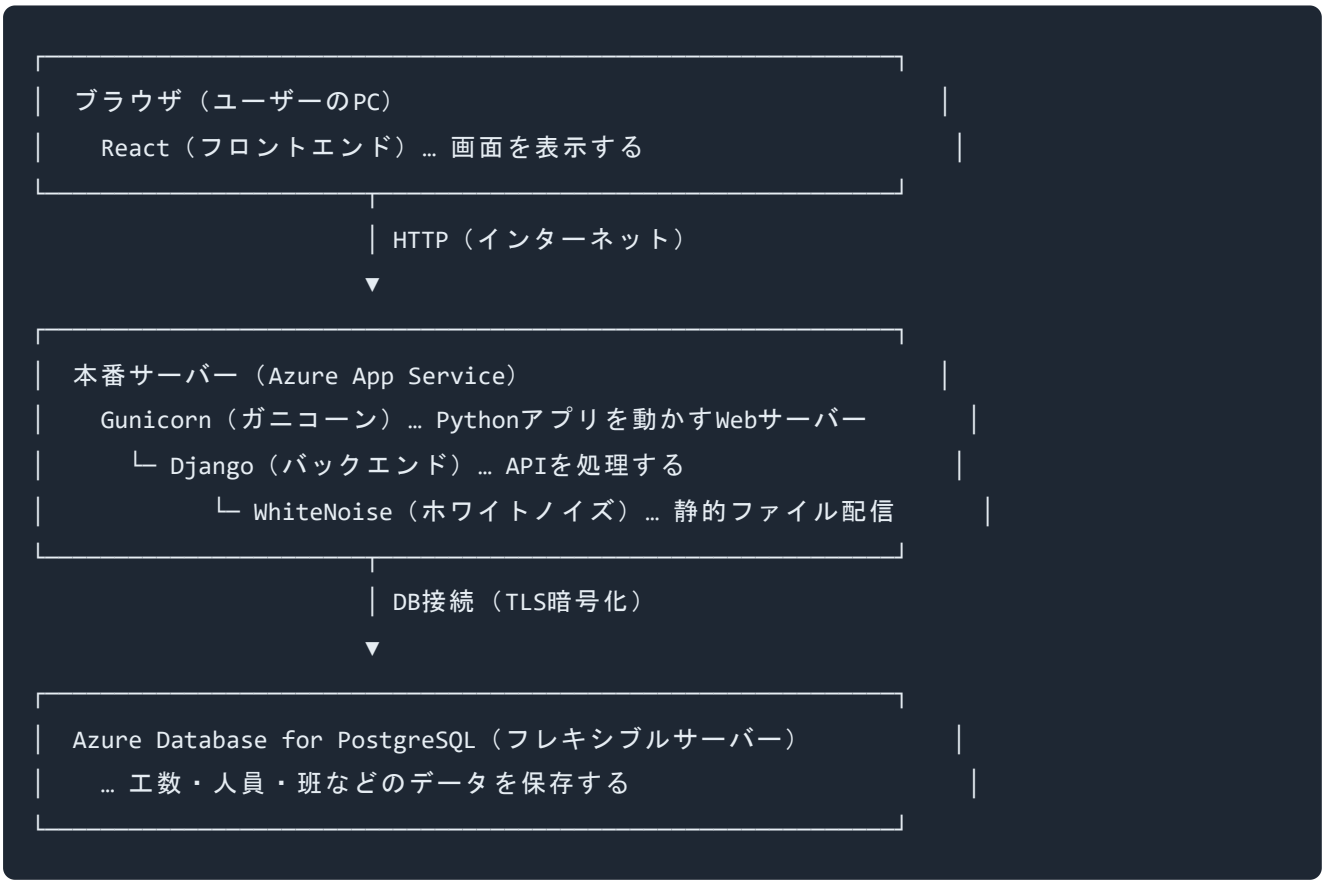
1.

e a i

ai e a ce

de l

1.1



1.2

> P h

---

2. 3

3

> e i e

2.1 3

		`localhost`		
				X

>

⚠️ PC A e

2.2

`.env`		
`frontend/.env.production`		
`hozen_another/settings.py`		`ALLOWED_HOSTS`

`.env` `KEY=値`
`.gitignore`
`settings.py`
`env('SECRET_KEY')` `.env`

⚠️ `.env` Gi

3.

`CLAUDE.md`

> c a d
> `Enter`

3.1 Dja g

> :
> `http://localhost:8000`

```
# Djangoの開発用サーバーを localhost:8000 で起動する
python manage.py runserver
```

- `python`
- `manage.py`
- `runserver`



```
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

`http://localhost:8000`

`Ctrl + C`

> ⚠️ **`runserver`**

3.2 Reac

> : `localhost:3000`

```
# Reactの開発サーバーを localhost:3000 で起動（ファイル保存で自動反映）
npm start
# 本番用に最適化したファイル一式を frontend/build/ へ書き出す
npm run build
```

- `npm`
- `start`
- `build` `build`

> b ild
>
> `npm run build`
>
> `frontend/build/`

3.3

> ig a i
>
> `models.py`

> : `models.py`

```
# models.pyの変更から「変更の設計図」を作る
python manage.py makemigrations
# その設計図をデータベースに実際に適用する
python manage.py migrate
```

- `makemigrations` `kosu/migrations/`
- `migrate`

> ⚠️ `makemigrations` `migrate` `migrate`

> ⚠️

3.4

`django-q` `Q_CLUSTER`

> :

```
# django-qのワーカー（裏方の処理係）を起動する
python manage.py qcluster
```

- `qcluster`

ke

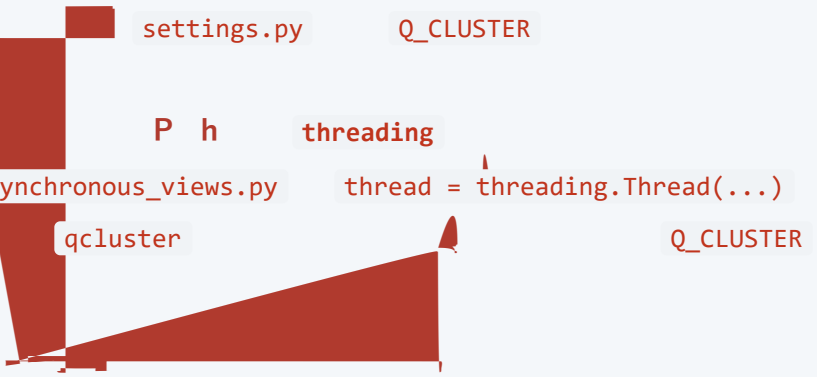
⚠️ settings.py Q_CLUSTER

django_q

dja g P h threading

asynchronous_views.py thread = threading.Thread(...)

qcluster Q_CLUSTER



3.5

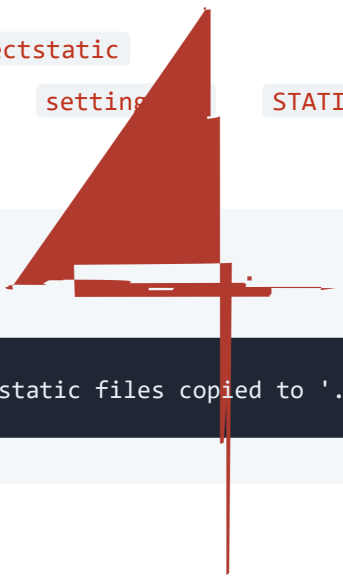
:

staticfiles/

```
# 静的ファイルを STATIC_ROOT（staticfiles/）へ集約する
python manage.py collectstatic
```

- collectstatic

setting STATIC_ROOT staticfiles/



:

```
XXX static files copied to '.../staticfiles'.
```

> ⚠️ `npm run build` `collectstatic`

3.6

`python manage.py runserver`		
`python manage.py migrate`		
`python manage.py makemigrations`		`models.py`
`python manage.py qcluster`		
`python manage.py collectstatic`		
`python manage.py createsuperuser`		
`python manage.py shell`		
`npm start`		
`npm run build`		
`npm run test:run`		

4.

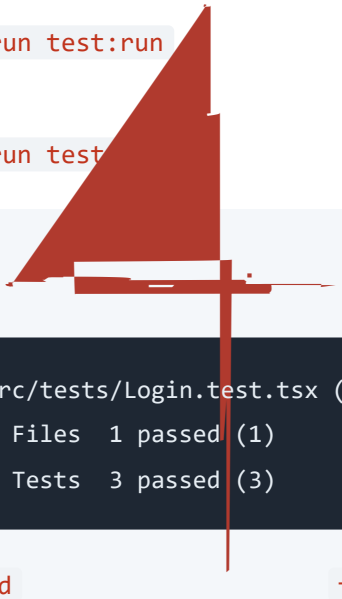
> e
>
> Vi e

4.1

> : `frontend/src/tests/`

```
# 監視モードでテスト（ファイル変更のたびに再実行）
npm test
# テストを1回だけ実行して終了（CI・コミット前向き）
npm run test:run
# ブラウザ上のUIでテスト結果を見る
npm run test:ui
```

- `npm test`
- `npm run test:run`
- `npm run test`



> ```
> ✓ src/tests/Login.test.tsx (3)
> Test Files  1 passed (1)
>      Tests  3 passed (3)
> ```
>
> `passed` `failed`

4.2

	`npm test`	
	`npm run test:run`	
	`npm run test:run`	

>

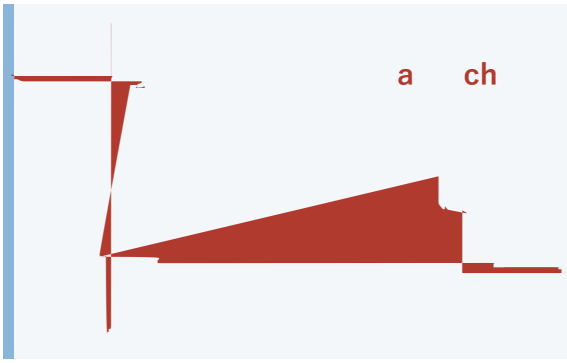
5.

5.1

> :

```
【バックアップ（ダウンロード）の流れ】

① ユーザーが「工数バックアップ」ボタンを押す
        |
        ▼
② フロント → POST /api/.../kosu_backup を送信（開始日・終了日つき）
        |
        ▼
③ バックエンド backup() … タスクID（受付番号）を発行し、別スレッドで処理を開始
        |   AsyncTask(status='pending') を作成 → task_id を即座に返す
        ▼
④ 別スレッドで generate_kosu_backup() が走る（tasks.py）
        |   DBからデータ取得 → openpyxlでExcel作成 → media/ に保存
        |   完了したら AsyncTask.status='success', result=ファイルパス
        ▼
⑤ フロントは task_id を使って「終わった？」を数秒おきに問い合わせ（ポーリング）
        |   GET /api/.../check_task_status?task_id=...
        ▼
⑥ status='success' になったら、file_path を使ってダウンロード
        |   GET /api/.../download_file?file_path=...
        ▼
⑦ ブラウザにExcelが保存される。サーバー側のファイルは3秒後に自動削除
```



lli g

check_task_status status

pending success error

5.2 3

`kosu/tasks.py`		
`kosu/views/asynchronous_views.py`		
`kosu/models.py` `AsyncTask`		

`tasks.py`

---

6. a k . e l

6.1

:

```
# ファイル・フォルダ操作（パス結合・存在確認・削除）
import os
# settings.py の値（MEDIA_ROOT等）を読むため
from django.conf import settings
# タイムゾーン付き
```

- `import pytz`
- `import datetime` `date`

```

```

```

```

```

```

```
# 工数データバックアップ非同期処理
# 開始日と終了日を受け取る関数
def generate_kosu_backup(start_day, end_day):
  # 新しいExcelブック作成
  # 空っぽのExcelブック（ファイル）を新規作成
  wb = openpyxl.Workbook()
  # その中の「最初のシート」を取り出す
  ws = wb.active
```

- `def generate_kosu_backup(start_day, end_day):` `def` `start_day` `end_day` `asynchronous_views.py`
- `wb = openpyxl.Workbook()` `wb`
- `ws = wb.active` `ws` `ws`

> e l 3
>
> `Workbook()` `wb.active` `ws.append([...])` `ws.cell(行, 列)`

```
# ヘッダー作成
headers = [
  # 1〜5列目の見出し
  '従業員番号', '氏名', '工数区分定義Ver', '就業日', '直',
  # 6〜9列目
  '作業内容', '作業詳細', '残業時間', '昼休憩時間',
  # 10〜12列目
  '残業休憩時間1', '残業休憩時間2', '残業休憩時間3',
  # 13〜15列目
  '就業形態', '工数入力OK_NG', '休憩変更チェック',
  ]
# この見出しリストをExcelの1行目として追加
ws.append(headers)
```

- `headers = [...]` `[ ]`
- `ws.append(headers)` `append`
  1

> ⚠️
>
> `load_kosu_file` `expected_headers`

```
# 工数データ取得
# 工数テーブルから条件で絞り込み
kosu_data = Business_Time_graph.objects.filter(
  # 就業日が start_day 以上 かつ end_day 以下
  work_day2__gte=start_day, work_day2__lte=end_day
  )
```

- `Business_Time_graph.objects.filter(...)` `Business_Time_graph` `objects` `.filter(...)`

- `work_day2__gte=start_day` `work_day2` `start_day` `__gte` g e
- `work_day2__lte=end_day` `work_day2` `end_day` `__lte` l e
- AND `start_day` ≦ 就業日 ≦ `end_day`

> Q e Se `.filter()` `for` `.exists()`

```
# データ書き込み
# 取得した工数データを1件ずつ取り出す
for item in kosu_data:
  # Excel1行ぶんのデータをリストにまとめる
  row = [
    # 従業員番号・氏名・Ver・就業日
    item.employee_no3, str(item.name), item.def_ver2, item.work_day2,
    # 直・作業内容・作業詳細・残業時間
    item.tyoku2, item.time_work, item.detail_work, item.over_time,
    # 昼休憩・残業休憩1・残業休憩2
    item.breaktime, item.breaktime_over1, item.breaktime_over2,
    # 残業休憩3・就業形態・入力OK_NG
    item.breaktime_over3, item.work_time, item.judgement,
    # 休憩変更チェック
    item.break_change,
    ]
  # この1行をExcelシートの末尾に追加
  ws.append(row)
```

- `str(item.name)` `item.name` `str(...)`

- `ws.append(row)` `for`

> **`name` `str()`** `employee_no3` `name`
>
> `str()` `__str__`

```
# 保存先のディレクトリ確認・作成
# settings.pyのMEDIA_ROOT（media/フォルダのパス）を取得
media_dir = settings.MEDIA_ROOT
# もしそのフォルダが存在しなければ
if not os.path.exists(media_dir):
```

```
    # ファイル名作成と保存
    # ファイル名を組み立て
    filename = f'工数データバックアップ_{start_day}_{end_day}.xlsx'
    # フォルダパスとファイル名を結合して完全なパスに
    filepath = os.path.join(media_dir, filename)
    # Excelブックをそのパスに保存（ここで初めてディスクに書かれる）
    wb.save(filepath)

    # ファイルパスを返却
    # 保存したファイルの場所を呼び出し元へ返す
    return filepath
```

- `filename = f'...{start_day}_{end_day}.xlsx'` `f'...'` f- i g `{}` `start_day=2026-05-01` `end_day=2026-05-31` `工数データバックアップ_2026-05-01_2026-05-31.xlsx`
- `filepath = os.path.join(media_dir, filename)` `\` `/` `+`
- `wb.save(filepath)`
- `return filepath` `handle_task` `AsyncTask.result`

> : `media/工数データバックアップ_2026-05-01_2026-05-31.xlsx`

> `tasks.py` `generate_member_backup` `generate_team_backup` `generate_def_backup`

---

7. a k . a da

generate_prediction

a da



7.1

```
# 工数区分定義予測データ出力非同期処理
# 開始日・終了日を受け取る
def generate_prediction(data_day, data_day2):
  # 期間内の工数データ取得
  # 工数テーブルを期間で絞る
  kosu_filter = Business_Time_graph.objects.filter(
    # data_day 以上 data_day2 以下
    work_day2__gte=data_day, work_day2__lte=data_day2
    )

  # Excelに書き出すためのデータを準備
  # 結果を貯める空のリストを用意
  data = []
```

- `def generate_prediction(data_day, data_day2):` `data_day` `data_day2`
- `kosu_filter = Business_Time_graph.objects.filter(...)`
- `data = []`

7.2 1

```
  # 取得した工数データを処理
  # 工数レコードを1件ずつ
  for kosu in kosu_filter:
    # 作業内容をリストに解凍
    # 作業内容（記号の文字列）を1文字ずつのリストに
    kosu_list = list(kosu.time_work)
    # 作業詳細をリストに解凍
    # 作業詳細を「$」で区切ってリストに
    detail_list = kosu.detail_work.split('$')
    # 工数定義区分取得
    # この工数のVerに対応する定義を取得
    def_filter = kosu_division.objects.filter(kosu_name=kosu.def_ver2)
```

- `for kosu in kosu_filter:` `kosu`
- `kosu_list = list(kosu.time_work)` `kosu.time_work` `AABBC` 1 `list(...)` `['A','A','B','B','C']`
- `detail_list = kosu.detail_work.split('$')` `kosu.detail_work` 部品交換$点検$$清掃 `$` `.split('$')` `['部品交換','点検','','清掃']`
- `def_filter = kosu_division.objects.filter(kosu_name=kosu.def_ver2)` `def_ver2` `A`

> li `$` `'$'.join(リスト)` `$`

7.3

```
    # リストの長さ取得
    # 2つのリストのうち長い方の長さ
    max_length = max(len(kosu_list), len(detail_list))
    # 1要素ごとにExcelに書き込み
    # 0 から max_length-1 まで繰り返す
    for i in range(max_length):
      # 工数定義区分がある場合の処理
      # 対応する定義が存在するなら
      if def_filter.exists():
        # 工数定義区分リスト作成
        # 記号→名前の対応リストを作る
        choices_list, def_n = kosu_division_dictionary(kosu.def_ver2)
```

- `max_length = max(len(kosu_list), len(detail_list))`
  `len(...)` `max(a, b)`

- `for i in range(max_length):` `0, 1, 2, ... max_length-1`
  `i`
- `if def_filter.exists():` `.exists()` `True/False`

- `choices_list, def_n = kosu_division_dictionary(kosu.def_ver2)`
  `choices_list` `def_n` `choices_list`
  `(記号, 名前)`

```
        # 作業内容を工数区分定義に変換
        # 対応表を1ペアずつ
        for k in choices_list:
          # 工数区分定義の記号と作業内容が同じ場合の処理
          # ペアの記号が、i番目の作業内容記号と一致したら
          if k[0] == kosu_list[i]:
            # 作業詳細が空欄でない場合の処理
            # i番目の作業詳細が空でないなら
            if detail_list[i] != '':
              # 作業内容と作業詳細をセットで定義
              row = [
                  # 区分名（範囲内なら名前、外なら空）
                  k[1] if i < len(kosu_list) else '',
                  # 作業詳細（同上）
                  detail_list[i] if i < len(detail_list) else '',
              ]
              # 作業内容と作業詳細を書き込み
              # 結果リストに1ペア追加
              data.append(row)
              #ループから抜ける
              # 一致したら内側ループを抜ける（無駄な探索をやめる）
              break
```

- `for k in choices_list:` `k` `(記号, 名前)`
- `if k[0] == kosu_list[i]:` `k[0]` `kosu_list[i]`

- `if detail_list[i] != '':`

- `row = [k[1] if ... else '', detail_list[i] if ... else '']` `k[1]` `detail_list[i]` `A if 条件 else B` `i`

- `data.append(row)` `data`
- `break` `choices_list` `for`

> **`A if 条件 else B`**
>
> `if/else` `x = '大人' if age >= 18 else '子供'`

7.4 a da

```
# DataFrameに変換
# リストを表(DataFrame)に変換
df = pd.DataFrame(data, columns=['工数定義区分', '作業詳細'])

# フィルタリング条件に基づいて不要な行を削除
# 区分が空・#・$ の行を除外
df = df[~df['工数定義区分'].isin(['', '#', '$'])]
# 作業詳細が空の行を除外
df = df[df['作業詳細'] != '']

# 工数定義区分と作業詳細の重複行を削除
# 同じ組み合わせは最初の1つだけ残す
df = df.drop_duplicates(subset=['工数 区分', '作業詳細'], keep='first')
```



- `df = pd.DataFrame(data, columns data`
  Da aF a e columns
- `df = df[~df['工数定義区分'].isin(['', '#', '$'])]` `isin([...])`
  ~ NOT
  '' # $ # $
- `df = df[df['作業詳細'] != '']`
- `df.drop_duplicates(subset=[...], keep='first')` `drop_duplicates`
  1
  `keep='first'`

> Da aF a e
> df[条件]
> `df.drop_duplicates()` for

7.5 l a da

```
# 保存先のディレクトリ確認・作成
# 保存先フォルダ
media_dir = settings.MEDIA_ROOT
# 無ければ
if not os.path.exists(media_dir):
  # 作る
  os.makedirs(media_dir)

# ファイル名作成と保存
# ファイル名
filename = f'工数予測データバックアップ_{data_day}_{data_day2}.xlsx'
# 完全なパス
filepath = os.path.join(media_dir, filename)

# Excelファイルをメモリ上で作成して保存
# ExcelWriterを開く
with pd.ExcelWriter(filepath, engine='openpyxl') as writer:
    # DataFrameをExcelに書き出す（行番号は出さない）
    df.to_excel(writer, index=False)

# ファイルパスを返却
# 保存パスを返す
return filepath
```



-
- `with pd.ExcelWriter(filepath, engine='openpyxl') as writer:`

  `engine='openpyxl'`

  `with ... as ...:`

- `df.to_excel(writer, index=False)`

  `index=False`

> 6 `wb.save` 7 `ExcelWriter`
>
> `wb.save()`
>
> `to_excel`
>
> e l E cel

---

8. a k . E cel DB

E cel DB

8.1 `delete_kosu_data`

:

```
# 工数データ削除非同期処理
# 開始日・終了日を受け取る
def delete_kosu_data(start_day, end_day):
  # 工数データ取得
  # 期間で絞る
  kosu_obj = Business_Time_graph.objects.filter(work_day2__gte=start_day,
work_day2__lte=end_day)
  # 取得した工数データを削除
  # 絞った行をまとめて削除
  kosu_obj.delete()

  # 戻り値なし（成功扱い）
  return None
```

- `kosu_obj = ...filter(...)`
- `kosu_obj.delete()` `.delete()`
- `return None` `handle_task`

⚠️ `.delete()`
 `validate_dates`

8.2 `load_kosu_file`

:

```
# 工数データロード非同期処理
# アップロードされた一時ファイルのパスを受け取る
def load_kosu_file(file_path):
  # 以下の処理でエラーが出ても落ちないように包む
  try:
    # 一時ファイルパスを変数に保持
    temp_file_path = file_path
    # Excelファイルを開く
    wb = openpyxl.load_workbook(temp_file_path)
    # 最初のシートを取得
    ws = wb.worksheets[0]
```

- `def load_kosu_file(file_path):` `asynchronous_views.py`
- `try:` `except`
- `wb = openpyxl.load_workbook(temp_file_path)` `load_workbook` `Workbook()`
- `ws = wb.worksheets[0]` `worksheets[0]` `wb.active`

```
    # ヘッダー定義
    expected_headers = [
      '従業員番号', '氏名', '工数区分定義Ver', '就業日', '直',
      '作業内容', '作業詳細', '残業時間', '昼休憩時間',
      '残業休憩時間1', '残業休憩時間2', '残業休憩時間3',
      '就業形態', '工数入力OK_NG', '休憩変更チェック',
    ]

    # 実ファイルの1行目を取得
    actual_headers = [ws.cell(1, col).value for col in range(1, len(expected_headers) +
1)]

    # ヘッダーが一致しない場合、一時ファイルを削除しエラーを返す
    # 期待する見出しと違ったら
    if actual_headers != expected_headers:
      # 一時ファイルを消して
      os.remove(temp_file_path)
      # エラーを返す
      return {'status': 'error', 'message': '無効なファイルフォーマットです。'}, None
```

- `expected_headers = [...]`
- `actual_headers = [w ol).value for col in range(1, len(expected_headers) + 1)]` `range(1, 16)` `ws.cell(1, col).value`
- `if actual_headers != expected_headers:`
- `os.remove(temp_file_path)`
- `return {'status': 'error', .}, None` 2
  `{'status': 'error', ...}` `None` `handle_task` `status='error'`

> li c ebe i `[式 for 変数 in 範囲]`
> `[ws.cell(1, col).value for col in range(1, 16)]`
> `for` `append`

`ws.cell(行, 列)` 1
`ws.cell(1, 1)` `ws.cell(2, 3)` `.value`

```
    # 4. データ読み込みとDB保存
    # 2行目から最終行まで（1行目は見出しなので飛ばす）
    for i in range(2, ws.max_row + 1):
      # 1列目＝従業員番号
      employee_no = ws.cell(row=i, column=1).value
      # 4列目＝就業日
      work_day2 = ws.cell(row=i, column=4).value

      try:
        # その従業員番号の人員を取得
        member_instance = member.objects.get(employee_no=employee_no)
      # 見つからなかったら
      except ObjectDoesNotExist:
        print(f"警告: 従業員番号 {employee_no} (行 {i}) のメンバーが見つかりませんでした。スキップします。")
        # 次のループへ進む            # この行は飛ばして次へ
        continue
```

- `for i in range(2, ws.max_row + 1):` `i` 2 `ws.max_row` 2 +1
- `employee_no = ws.cell(row=i, column=1).value`
- `work_day2 = ws.cell(row=i, column=4).value`
- `member_instance = member.objects.get(employee_no=employee_no)` `.get(...)` 1
- `except ObjectDoesNotExist:` `.get()`
- `print(...)` `f"..."`
- `continue` `continue`

> **.get()** **.filter()** .filter() **0** **OK**
> .get() **1** DoesNotExist
> MultipleObjectsReturned .get()
> .filter()

```
# もし既に同一データが存在するなら削除
existing_data = Business_Time_graph.objects.filter(
  # 同じ人・同じ日の既存データ
  employee_no3=employee_no, work_day2=work_day2
  )
# 既存があれば
if existing_data.exists():
  # 消す（重複防止）
  existing_data.delete()
```

- `existing_data = ...filter(employee_no3=employee_no, work_day2=work_day2)`

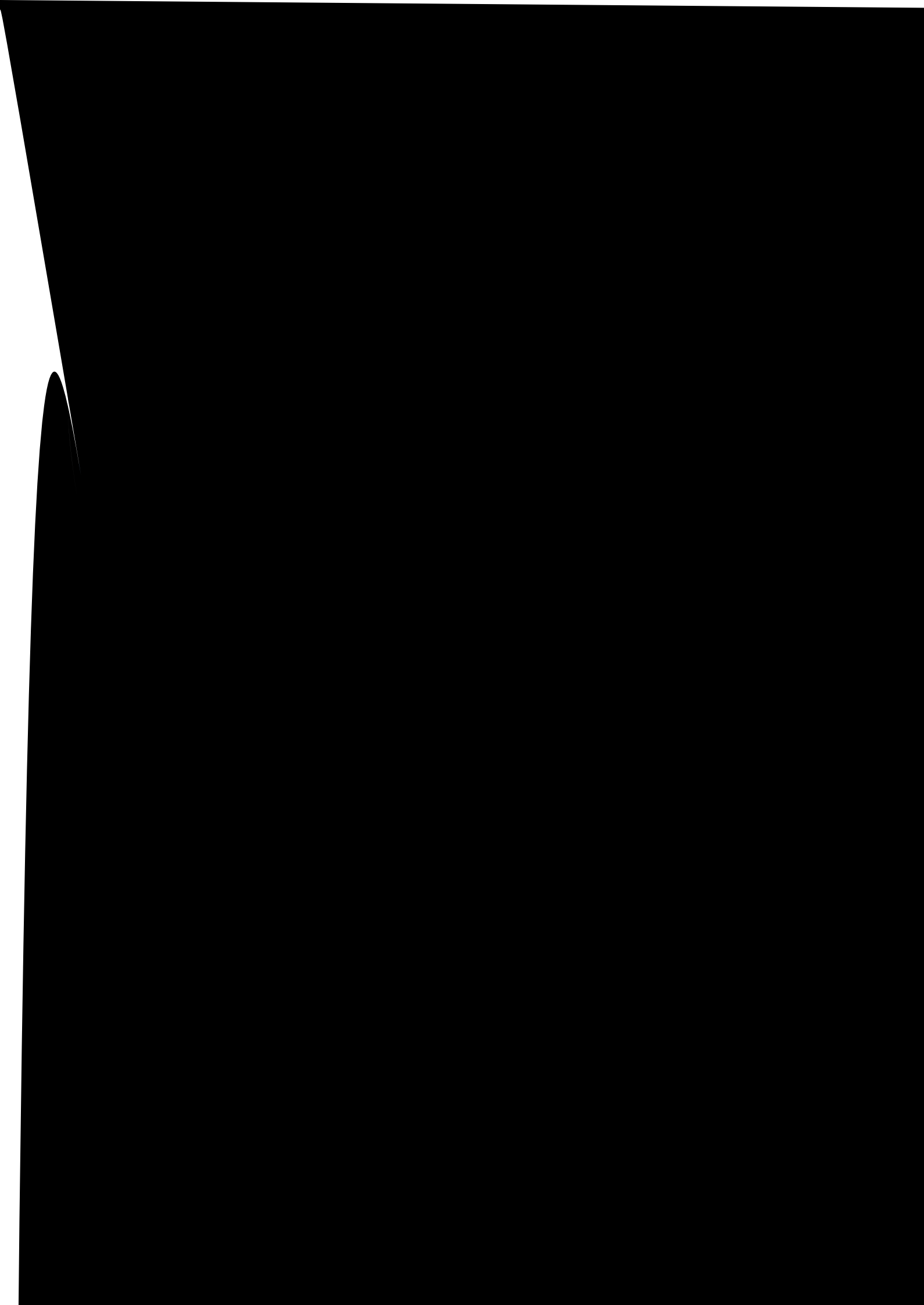
- `if existing_data.exists():`
- `existing_data.delete()`

```
        # 新データをインスタンスとして作成してDBに保存
        try:
          # 工数テーブルに新しい行を作成・保存
          Business_Time_graph.objects.create(
            # 従業員番号
            employee_no3=employee_no,
            # 人員（先ほど取得したオブジェクトを紐付け）
            name=member_instance,
            # 3列目＝定義Ver
            def_ver2=ws.cell(row=i, column=3).value,
            # 就業日
            work_day2=work_day2,
            # 5列目＝直
            tyoku2=ws.cell(row=i, column=5).value,
            # 6列目＝作業内容
            time_work=ws.cell(row=i, column=6).value,
            # 7列目＝作業詳細
            detail_work=ws.cell(row=i, column=7).value,
            # 8列目＝残業時間
            over_time=ws.cell(row=i, column=8).value,
            # 9列目＝昼休憩
            breaktime=ws.cell(row=i, column=9).value,
            # 10列目＝残業休憩1
            breaktime_over1=ws.cell(row=i, column=10).value,
            # 11列目＝残業休憩2
            breaktime_over2=ws.cell(row=i, column=11).value,
            # 12列目＝残業休憩3
            breaktime_over3=ws.cell(row=i, column=12).value,
            # 13列目＝就業形態
            work_time=ws.cell(row=i, column=13).value,
            # 14列目＝入力OK_NG
            judgement=ws.cell(row=i, column=14).value,
            # 15列目＝休憩変更チェック
            break_change=ws.cell(row=i, column=15).value,
          )
        #[illegible]合性エラー（重複等）が起きたら
        ex[illegible]pt Integrity[illegible] create_err:
            [illegible]int(f"エラー: 行 {i} のデータ作成中に整合性エラーが発生しました:
{create_[illegible]r}。スキップします。")
            #[illegible]次のループへ進む            # その行を飛ばす
            c[illegible]ntinue
```

- `Business_Time_graph.objects.create(...)` `.create(...)` DB
- `name=member_instance` `.get()`

- `ws.cell(row=i, column=N).value`
-



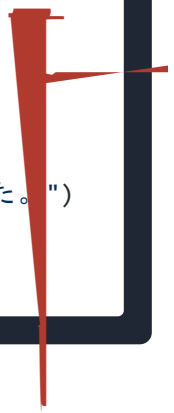


> `strptime` `f`

9.2 `load_member_file`

```
    for i in range(2, ws.max_row + 1):
      employee_no_value = ws.cell(row=i, column=1).value

      # 既に同じ番号の人員がいたら
      if member.objects.filter(employee_no=employee_no_value).exists():
        print(f"従業員番号 {employee_no_value} は既に存在するためスキップしました。")
        # 上書きせず飛ばす
        continue
```



-

- `if member.objects.filter(...).exists():`
  `continue`
-

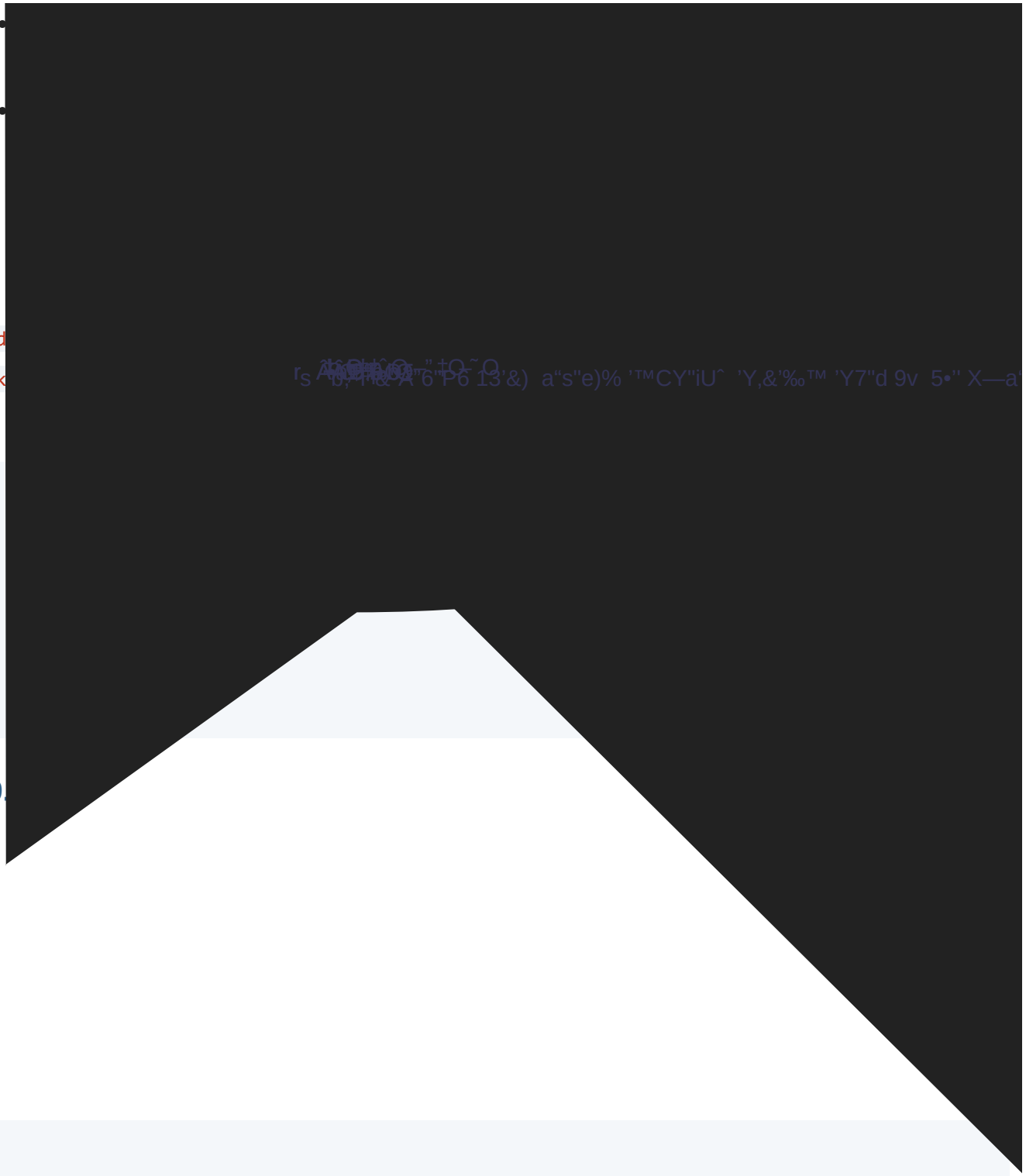


9.3 `generate_team_backup` / `load_team_file`

- `team_member`
-

9.4 `generate_def_backup` / `load_def_file`

```
  # ヘッダー作成
  headers = ['工数区分定義Ver名'] + [item for i in range(1, 51) for item in [f'工数区分名{i}', f'定義{i}', f'作業内容{i}', f'変動/固定{i}']]
```



9.6

`generate_AsyncTask_backup` `generate_History_backup`

```
# 文字列を日付に変換
start_date = datetime.datetime.strptime(start_day, '%Y-%m-%d').date()
end_date = datetime.datetime.strptime(end_day, '%Y-%m-%d').date()
```

- `strptime(start_day, '%Y-%m-%d')` `strftime` `'2026-05-01'` `.date()`

```
for item in AsyncTask_data:
  try:
    # settings.TIME_ZONE（Asia/Tokyo）のタイムゾーン
    target_tz = pytz.timezone(settings.TIME_ZONE)
    # UTC→日本時間に変換しタイムゾーン情報を外す
    created_at_local = item.created_at.astimezone(target_tz).replace(tzinfo=None)
    updated_at_local = item.updated_at.astimezone(target_tz).replace(tzinfo=None)
  except ImportError:
    # 万一変換できなければそのまま
    created_at_local = item.created_at.replace(tzinfo=None)
    updated_at_local = item.updated_at.replace(tzinfo=None)
```

- `target_tz = pytz.timezone(settings.TIME_ZONE)` `settings.TIME_ZONE` `'Asia/Tokyo'`

- `item.created_at.astimezone(target_tz)`

+9

- `.replace(tzinfo=None)`

UTC

`USE_TZ = True` UTC

⚠️ `astimezone`

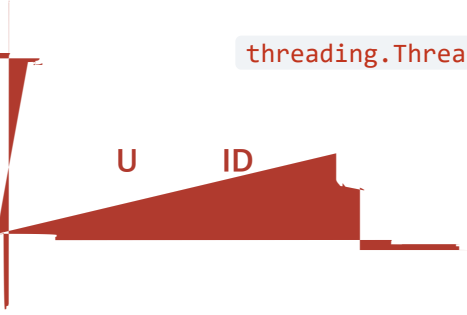
---

10. a ch _ ie .

`asynchronous_views.py`

10.1

```
# ファイル操作
import os
# 別スレッドを起動する道具（非同期の要）
import threading
# 一意なID（タスクID）を発行する道具
import uuid
# 日付処理（バリデーション用）
import datetime
# sleep（待ち）に使う
import time
# 一時ファイルを安全に作る道具
import tempfile
from ..tasks import generate_kosu_backup, delete_kosu_data, load_kosu_file, \
                    generate_member_backup, load_member_file, generate_team_backup,
load_team_file, \
                    generate_def_backup, load_def_file, generate_choice_backup,
load_choice_file, \
                    generate_inquiry_backup, load_inquiry_file,
generate_setting_backup, load_setting_file, \
                    generate_AsyncTask_backup, delete_AsyncTask_data,
generate_History_backup ,delete_History_data
# タスク状態を保存するモデル
from ..models import AsyncTask
# HTTPステータスコードの定数（status.HTTP_400_...）
from rest_framework import status
# API用デコレータ
from rest_framework.decorators import api_view, parser_classes
# リクエストの解析方式
from rest_framework.parsers import MultiPartParser, JSONParser, FormParser
# URLからurl_nameを逆引きする道具
from django.urls import resolve
# JSON応答・ファイル応答
from django.http import JsonResponse, FileResponse
```



- `import threading`　　`threading.Thread(...)`
- `import uuid`　　U　　U　　ID
- `import tempfile`
- `from ..tasks import ...`　　`..tasks`　　`..`

`kosu/`　　`tasks.py`

- `from ..models import AsyncTask`
- `from rest_framework import status` `status.HTTP_400_BAD_REQUEST`
- `api_view, parser_classes`
- `resolve` `url_name`
- `JsonResponse, FileResponse`

> dec a `@名前`
> `@api_view(['POST'])`

10.2 `backup` ID

```
# POSTリクエストのみ受け付けるAPI
@api_view(['POST'])
# ファイルもJSONもフォームも解析できる
@parser_classes([MultiPartParser, JSONParser, FormParser])
# リクエストを受け取る
def backup(request):
  # タスクID生成
  # 世界で一意なIDを生成して文字列に
  task_id = str(uuid.uuid4())
  # 「処理中」状態でタスクを記録
  AsyncTask.objects.create(task_id=task_id, status='pending')

  # リクエストから開始日を取得
  start_day = request.data.get('start_day')
  # 終了日を取得
  end_day = request.data.get('end_day')
```

- `@api_view(['POST'])` `backup` POST
- `@parser_classes([...])`

- `task_id = str(uuid.uuid4())` `uuid.uuid4()` `str()` `'3f2a...e9'`
- `AsyncTask.objects.create(task_id=task_id, status='pending')`



```
# 工数バックアップなら
if url_name == 'kosu_backup':
  # 日付を検証
  error_response = validate_dates(start_day, end_day)
  # 検証エラーがあれば
  if error_response:
    # そのエラーを即返す
    return error_response
  # 実行する関数を「工数バックアップ」に決定
  task_function = generate_kosu_backup
  # その関数に渡す引数
  args = (start_day, end_day)
# 工数削除なら
elif url_name == 'kosu_delet':
  error_response = validate_dates(start_day, end_day)
  if error_response:
    return error_response
  # 「工数削除」関数に決定
  task_function = delete_kosu_data
  args = (start_day, end_day)
```

- `if url_name == 'kosu_backup':`
- `error_response = validate_dates(start_day, end_day)`
- `if error_response: return error_response`
- `task_function = generate_kosu_backup`
  `generate_kosu_backup()` `generate_kosu_backup`
- `args = (start_day, end_day)`

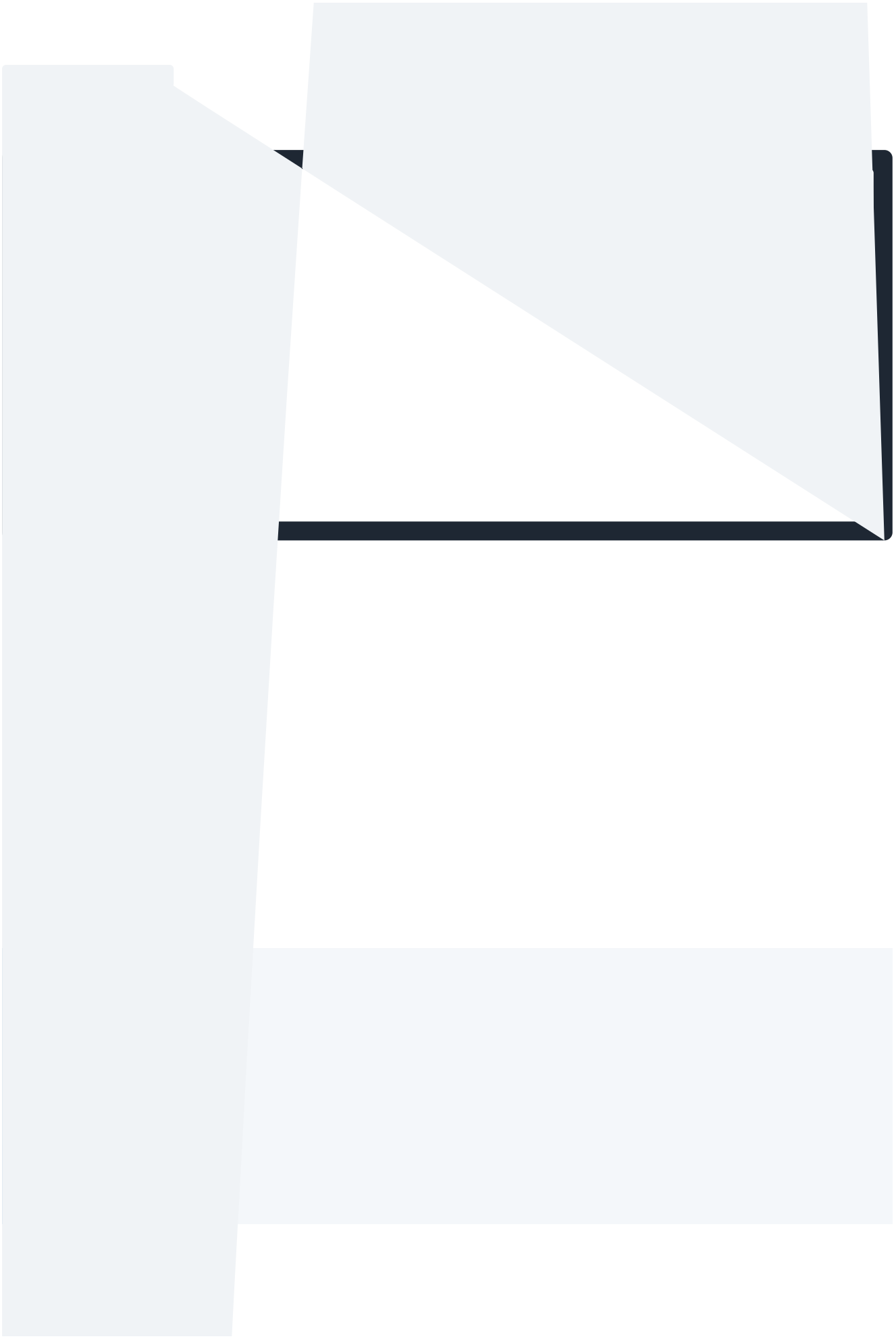
> `f = generate_kosu_backup` `f(...)`
> `task_function`

```
  # 工数ロードなら
  elif url_name == 'kosu_load':
    # アップロードされたファイルを取得
    kosu_file = request.FILES.get('file')
    # 一時パスを初期化
    temp_file_path = None
    try:
      # 一時ファイルを作成
      with tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False) as temp_file:
        # アップロードを小分け（チャンク）で
        for chunk in kosu_file.chunks():
          # 一時ファイルに書き込む
          temp_file.write(chunk)
        # 一時ファイルのパスを記憶
        temp_file_path = temp_file.name
      # 実行関数を「工数ロード」に
      task_function = load_kosu_file
      # その一時パスを引数に
      args = (temp_file_path,)
    except Exception as e:
      # 失敗したら一時ファイルを
      if temp_file_path and os.path.exists(temp_file_path):
        # 消す
        os.remove(temp_file_path)
      return JsonResponse({'status': 'error', 'message': f'ファイル書き込みエラー:
{str(e)}'}, status=500)
```

- `kosu_file = request.FILES.get('file')`
  `request.FILES`
- `with tempfile.NamedTemporaryFile(suffix=".xlsx", delete=False) as temp_file:`
  `suffix=".xlsx"` `delete=False`
  `tasks.py`
- `for chunk in kosu_file.chunks():`

- `temp_file.write(chunk)`
- `temp_file_path = temp_file.name`
  `load_kosu_file`
- `except`

`elif` `def_load` `member_load` `team_load` `choice_load` `inquiry_load` `setting_load` `load_*`



10.3 `validate_dates`

```
# 日付バリデーション関数
# 開始日・終了日を検証
def validate_dates(start_day, end_day):
  # 今日を文字列で
  today_str = datetime.date.today().strftime('%Y-%m-%d')
  # どちらか空なら
  if not start_day or not end_day:
    return JsonResponse({'status': 'error', 'message': '日付を指定してください。'},
status=status.HTTP_400_BAD_REQUEST)

  try:
    # 文字列を日付に
    end_date_obj = datetime.date.fromisoformat(end_day)
    today_date_obj = datetime.date.fromisoformat(today_str)
    start_date_obj = datetime.date.fromisoformat(start_day)
  # 変換できない＝形式不正なら
  except ValueError:
    return JsonResponse({'status': 'error', 'message': '日付の形式が不正です。'},
status=status.HTTP_400_BAD_REQUEST)
  # 終了日が今日以降なら
  if end_date_obj >= today_date_obj:
    return JsonResponse({'status': 'error', 'message': '昨日の日付までしか指定できませ
ん。'}, status=status.HTTP_400_BAD_REQUEST)
  # 開始日が終了日より後なら
  if start_date_obj > end_date_obj:
    return JsonResponse({'status': 'error', 'message': '開始日が終了日を超えていま
す。'}, status=status.HTTP_400_BAD_REQUEST)
  # 問題なければ None（エラーなし）を返す
  return None
```

- `if not start_day or not end_day:` `None`
- `datetime.date.fromisoformat(end_day)` `'2026-05-31'` `ValueError` `except`
- `if end_date_obj >= today_date_obj:`
- `if start_date_obj > end_date_obj:`
- `return None` `None` `backup` `None`

10.4 `check_task_status`

```
# GETで呼ぶAPI
@api_view(['GET'])
def check_task_status(request):
  # クエリ文字列からタスクIDを取得
  task_id = request.GET.get('task_id')

  # タスクIDがない場合、エラーを返す
  if not task_id:
    return JsonResponse({'status': 'error', 'message': 'タスクIDが指定されていません。'}, status=status.HTTP_400_BAD_REQUEST)

  try:
    # データベースからタスクIDに対応する状態を取得し返す
    # その受付番号のタスクを取得
    task = AsyncTask.objects.get(task_id=task_id)
    # 成功していれば
    if task.status == 'success':
      # 成功＋ファイルパスを返す
      return JsonResponse({'status': 'success', 'file_path': task.result})
    # 失敗していれば
    elif task.status == 'error':
      # エラーメッセージを返す
      return JsonResponse({'status': 'error', 'message': task.result})
    # まだ処理中なら
    else:
      # 「処理中(202)」を返す
      return JsonResponse({'status': 'pending'}, status=202)

  # その受付番号が存在しなければ
  except AsyncTask.DoesNotExist:
    return JsonResponse({'status': 'error', 'message': '無効なタスクIDです。'}, status=status.HTTP_400_BAD_REQUEST)
```

- `task_id = request.GET.get('task_id')` `?task_id=...`
- `task = AsyncTask.objects.get(task_id=task_id)`

- `if task.status == 'success':`

- `elif task.status == 'error':`
- `else: return JsonResponse({'status': 'pending'}, status=202)` `pending`
  202

- `except AsyncTask.DoesNotExist:`

> 202 Acce ed

> :

```
フロント: GET check_task_status?task_id=abc → 202 pending （まだ）
  （2秒待つ）
フロント: GET check_task_status?task_id=abc → 202 pending （まだ）
  （2秒待つ）
フロント: GET check_task_status?task_id=abc → 200 success + file_path （完了！）
  → ダウンロードへ
```

10.5 `download_file`

```
@api_view(['GET'])
def download_file(request):
  # 渡されたファイルパス
  file_path = request.GET.get('file_path')
  # そのファイルをバイナリ読み込みで開く
  file_handle = open(file_path, 'rb')
  # ダウンロード応答を作る
  response = FileResponse(file_handle, as_attachment=True,
filename=os.path.basename(file_path))
```

- `file_path = request.GET.get('file_path')` `check_task_status`

- `file_handle = open(file_path, 'rb')` `'rb'` `b`
- `FileResponse(file_handle, as_attachment=True, filename=...)` `as_attachment=True` `filename=os.path.basename(file_path)`

```
# 遅延削除を行う内部関数
def delayed_file_cleanup():
    # 3秒待つ（ダウンロード完了を待つ）
    time.sleep(3)
    # ファイルがまだあれば
    if os.path.exists(file_path):
      try:
        # 削除
        os.remove(file_path)
      except Exception as e:
        # 失敗してもログだけ
        print(f"Cleanup failed after delay for {file_path}: {e}")

# 応答が閉じられたとき呼ばれる関数
def cleanup_on_close():
    # 削除を別スレッドで
    thread = threading.Thread(target=delayed_file_cleanup)
    thread.start()

# 応答の「閉じる」処理を差し替え
response.close = cleanup_on_close

# 応答を返す（ダウンロード開始）
return response
```

- `def delayed_file_cleanup():` `time.sleep(3)`
- `if os.path.exists(file_path): os.remove(file_path)` `try/except`
- `def cleanup_on_close():`
- `response.close = cleanup_on_close`
- `return response`

```
3
```

10.6 `handle_task`

`backup`

```python
# 非同期タスク処理（汎用版）
# 受付番号・実行関数・引数を受け取る
def handle_task(task_id, task_function, *args, **kwargs):
  try:
    # タスク関数を実行し、結果を取得
    # 選ばれたタスク関数を実際に実行
    result = task_function(*args, **kwargs)
    # 戻り値が「明示的なエラー」かを判定
    is_explicit_error = (
      # 結果がタプルで
      isinstance(result, tuple) and
      # 要素があり
      len(result) > 0 and
      # 最初の要素が辞書で
      isinstance(result[0], dict) and
      # status が 'error' なら
      result[0].get('status') == 'error'
    )

    # 該当タスクを取得
    task = AsyncTask.objects.get(task_id=task_id)

    # 明示エラーの場合
    if is_explicit_error:
      error_dict = result[0]
      # 状態をエラーに
      task.status = 'error'
      # メッセージ保存
      task.result = error_dict.get('message', 'タスク関数が明示的なエラーを返しました。')
    # 正常終了の場合
    else:
      # 状態を成功に
      task.status = 'success'
      # 結果（ファイルパス）を保存
      task.result = result

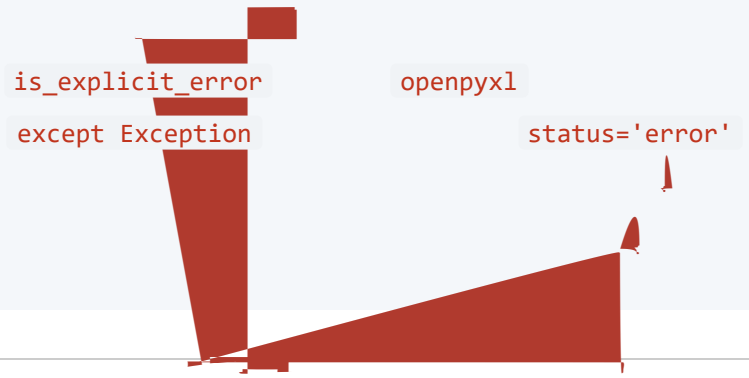
    # データベースに保存
    # 変更をDBに書き込む
    task.save()

  # 予期せぬエラーが出たら
  except Exception as e:
    try:
      task = AsyncTask.objects.get(task_id=task_id)
      # 状態をエラーに
      task.status = 'error'
      # エラー内容を保存
```

```
        task.result = str(e)
        task.save()
    # タスク自体が無ければ
    except AsyncTask.DoesNotExist:
        # ログだけ
        print(f"Error: AsyncTask with id {task_id} not found.")
```

- `def handle_task(task_id, task_function, *args, **kwargs):`
  `*args` `**kwargs`
- `result = task_function(*args, **kwargs)`
  `generate_kosu_backup(start_day, end_day)` `result`
- `is_explicit_error = (...)` `({'status':'error', ...}, None)`
  `isinstance(x, 型)`
- `task = AsyncTask.objects.get(task_id=task_id)`
- `if is_explicit_error:` `'error'`
- `else:` `'success'`
- `task.save()`
  `check_task_status`
- `except Exception as e:`

> 2
> `is_explicit_error` `openpyxl`
> `except Exception` `status='error'`



11. e i g .

`settings.py`

11.1

```
# ファイルパスを扱う近代的な道具
from pathlib import Path
# OS機能（環境変数等）
import os
# .envファイルを読む外部ライブラリ
import environ
# ログを一定サイズで回転させるハンドラ
from logging.handlers import RotatingFileHandler

# プロジェクトの根っこフォルダのパス
BASE_DIR = Path(__file__).resolve().parent.parent

# 環境変数を読む道具を作成
env = environ.Env()
# ルートの.envファイルを読み込む
env.read_env(os.path.join(BASE_DIR, '.env'))

# 秘密鍵を.envから取得
SECRET_KEY=env('SECRET_KEY')

# デバッグモードを.envから真偽値で取得
DEBUG=env.bool('DEBUG')

# 既定の文字コードをUTF-8に
DEFAULT_CHARSET = 'utf-8'
```

- `from pathlib import Path` `/`
- `BASE_DIR = Path(__file__).resolve().parent.parent` `__file__` `settings.py` `.parent.parent` 2

- `env = environ.Env()` `.env`
- `env.read_env(os.path.join(BASE_DIR, '.env'))` `.env`
- `SECRET_KEY=env('SECRET_KEY')` `.env` Dja g CSRF `.env`

- `DEBUG=env.bool('DEBUG')` `True/False` `False` `True`
- `DEFAULT_CHARSET = 'utf-8'`

> ⚠️ `DEBUG=True`
>
> `.env` `DEBUG=False`

11.2 ALLOWED_HOSTS

```
ALLOWED_HOSTS = [
    # 本番Azureのドメイン
    'hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
    # ローカル開発用
    'localhost',
    # ローカル開発用（IPアドレス表記）
    '127.0.0.1'
    ]
```

- `ALLOWED_HOSTS = [...]`
-
- `400 Bad Request`

> H
>
> URL

11.3 INSTALLED_APPS / MIDDLEWARE

```
INSTALLED_APPS = [
    # 管理画面
    'django.contrib.admin',
    # 認証
    'django.contrib.auth',
    # コンテンツタイプ
    'django.contrib.contenttypes',
    # セッション
    'django.contrib.sessions',
    # メッセージ
    'django.contrib.messages',
    # 静的ファイル
    'django.contrib.staticfiles',
    # Bootstrap4
    'bootstrap4',
    # 日付ピッカー
    'bootstrap_datepicker_plus',
    # 本アプリ本体
    'kosu',
    # 非同期タスクキュー
    'django_q',
    # REST API
    'rest_framework',
    # CORS（クロスオリジン）対応
    'corsheaders',
]
```

- `INSTALLED_APPS`
- `'kosu'` `models.py` `views/`
- `'django_q'`
- `'rest_framework'`
- `'corsheaders'`

```
MIDDLEWARE = [
    # CORS処理（先頭に近い位置が必要）
    'corsheaders.middleware.CorsMiddleware',
    # セキュリティ
    'django.middleware.security.SecurityMiddleware',
    # セッション
    'django.contrib.sessions.middleware.SessionMiddleware',
    # 共通処理
    'django.middleware.common.CommonMiddleware',
    # CSRF対策
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',# 認証
    # メッセージ
    'django.contrib.messages.middleware.MessageMiddleware',
    # クリックジャッキング対策
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    # 静的ファイル配信
    'whitenoise.middleware.WhiteNoiseMiddleware',
    # 自作:現在リクエスト保持
    'kosu.middleware.clear_session_middleware.CurrentRequestMiddleware',
]
```

- `MIDDLEWARE`
- `'whitenoise.middleware.WhiteNoiseMiddleware'` Whi eN i e

- `'corsheaders.middleware.CorsMiddleware'`

-

> iddle a e

> Whi eN i e

11.4 CORS

```
# すべてのオリジンを許可（開発向け）
CORS_ORIGIN_ALLOW_ALL = True
# Cookie等の資格情報を許可
CORS_ALLOW_CREDENTIALS = True
CORS_ALLOW_ORIGINS = [
    # React開発サーバー
    'http://localhost:3000',
    # Django開発サーバー
    'http://localhost:8000',
    'http://127.0.0.1:8000',
]
```

- `CORS_ORIGIN_ALLOW_ALL = True`

- `CORS_ALLOW_CREDENTIALS = True`

> CORS C -O igi Re ce Sha i g
>
> `localhost:3000` `localhost:8000`

11.5 REST_FRAMEWORK /

```
REST_FRAMEWORK = {
    # ページ番号方式
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    # 1ページ20件
    'PAGE_SIZE': 20,
    'DEFAULT_PARSER_CLASSES': (
        # JSONを解析
        'rest_framework.parsers.JSONParser',
    ),
    'DEFAULT_RENDERER_CLASSES': (
        # JSONで返す
        'rest_framework.renderers.JSONRenderer',
    ),
}

# セッションをDBに保存
SESSION_ENGINE = 'django.contrib.sessions.backends.db'
# セッション有効期間（約10年=秒）
SESSION_COOKIE_AGE = 315360000

# CookieのSameSite属性
SESSION_COOKIE_SAMESITE = 'Lax'
# HTTPS必須にしない（開発向け）
SESSION_COOKIE_SECURE = False
# ルーティングの起点
ROOT_URLCONF = 'hozen_another.urls'
```



- `'PAGE_SIZE': 20`
- `SESSION_ENGINE = '...db'`
- `SESSION_COOKIE_AGE = 315360000`

- `SESSION_COOKIE_SAMESITE = 'Lax'` `SESSION_COOKIE_SECURE = False`
  `'None'` `True` HTTPS
  `Lax` `False`

> ⚠ HTTPS `SESSION_COOKIE_SECURE = True`

11.6 DATABASES

```
DATABASES = {
    'default': {
        # PostgreSQLを使う
        'ENGINE': 'django.db.backends.postgresql',
        # DB名（環境変数から）
        'NAME': os.getenv('DB_NAME', ''),
        # 接続ユーザー
        'USER': os.getenv('DB_USER', ''),
        # パスワード
        'PASSWORD': os.getenv('DB_PASSWORD', ''),
        # 接続先ホスト
        'HOST': os.getenv('DB_HOST', ''),
        # ポート番号（PostgreSQL既定）
        'PORT': '5432',
    }
}
```

- `'ENGINE': '...postgresql'` P g eSQL
- `'NAME'/'USER'/'PASSWORD'/'HOST'` `os.getenv('キー', '')` `''` `.env`
- `'PORT': '5432'`

> `.env` `DB_HOST`

> ⚠ `DB_HOST` `xxx.postgres.database.azure.com` DB `localhost` 1

11.7 Q_CLUSTER

```
# Django Q の設定
Q_CLUSTER = {
    # タスク保存にdefaultのDBを使う
    'orm': 'default',
    # ワーカー（働き手）を4つ
    'workers': 4,
    # 2000タスクごとにワーカーを再生成
    'recycle': 2000,
    # リトライまでの秒数
    'retry': 2000,
    # タスクのタイムアウト（30分）
    'timeout': 1800,
}
```

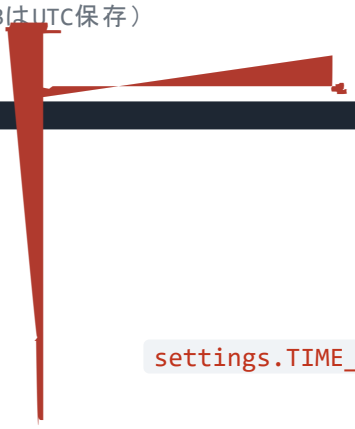
- `Q_CLUSTER = {...}` h eadi g

- `'orm': 'default'` `default`
- `'workers': 4`
- `'timeout': 1800`
- `'recycle': 2000`

11.8

```
AUTH_PASSWORD_VALIDATORS = [
    # ユーザー情報と似たPW禁止
    {'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
    # 最低長
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator'},
    # よくあるPW禁止
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
    # 数字だけ禁止
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},
]

# 表示言語を日本語に
LANGUAGE_CODE = 'ja'
# タイムゾーンを日本時間に
TIME_ZONE = 'Asia/Tokyo'
# 国際化を有効に
USE_I18N = True
# 時刻をタイムゾーン付きで扱う（DBはUTC保存）
USE_TZ = True
```



- `AUTH_PASSWORD_VALIDATORS`

- `LANGUAGE_CODE = 'ja'`
- `TIME_ZONE = 'Asia/Tokyo'` `settings.TIME_ZONE`
- `USE_TZ = True` DB UTC

11.9 Whi eN i e CSRF

```
STATICFILES_DIRS = [
    # Reactビルドの静的ファイル置き場
    os.path.join(BASE_DIR, 'frontend/build/static'),
]

# Reactのbuild内のindex.htmlをテンプレートとして指定
# React のindex.htmlを探す場所
TEMPLATES[0]['DIRS'] = [os.path.join(BASE_DIR, 'frontend/build')]
```

- `STATICFILES_DIRS`
  `frontend/build/static`
- `TEMPLATES[0]['DIRS'] = [...]` `index.html`

1 Dja g API Reac

```
# 静的ファイルのURL接頭辞
STATIC_URL = '/static/'
# 主キーの既定型
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
# collectstaticの集約先
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')

CSRF_TRUSTED_ORIGINS = [
  # 本番（HTTPS）
  'https://hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
  # ローカル
  'http://localhost:8000',
  'http://127.0.0.1:8000',
]

# WhiteNoiseの圧縮保存方式
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'

# バックアップファイル保存先指定
# メディアファイルのURL接頭辞
MEDIA_URL = '/media/'
# バックアップExcelの保存先（tasks.pyで使用）
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

- `STATIC_URL = '/static/'`
- `STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')` `collectstatic`

- `CSRF_TRUSTED_ORIGINS = [...]` CSRF

  `ALLOWED_HOSTS`
- `STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'`

  g i

  `.gz`

- `MEDIA_ROOT = os.path.join(BASE_DIR, 'media')` 6 9 **`settings.MEDIA_ROOT`**

  `media/`

CSRF C -Si e Re e F ge

`CSRF_TRUSTED_ORIGINS`

⚠ URL 3 `ALLOWED_HOSTS`
`CSRF_TRUSTED_ORIGINS` `frontend/.env.production`

11.10 LOGGING R a i gFileHa dle

```
LOGGING = {
    # 設定スキーマのバージョン（必ず1）
    'version': 1,
    # 既存ロガーを無効化しない
    'disable_existing_loggers': False,
    # ログの書式定義
    'formatters': {
        'simple': {
            # メッセージ本文のみ出力
            'format': '{message}',
            # {}スタイルの書式
            'style': '{',
        },
    },
    # ログの出力先定義
    'handlers': {
        'web_console': {
            # INFO以上を記録
            'level': 'INFO',
            # サイズで回転するファイルハンドラ
            'class': 'logging.handlers.RotatingFileHandler',
            # 上のsimple書式を使う
            'formatter': 'simple',
            # 出力ファイル名
            'filename': 'web_console.log',
            # 最大ファイルサイズ（1MB）
            'maxBytes': 1024 * 1024 * 1,
            # 古いログを1世代だけ残す
            'backupCount': 1,
            # 文字コード
            'encoding': 'utf-8',
        },
    },
    # どのロガーがどのハンドラを使うか
    'loggers': {
        'views_logger': {
            # web_consoleハンドラに出す
            'handlers': ['web_console'],
            # INFO以上
            'level': 'INFO',
            # 親ロガーに伝播させない
            'propagate': False,
        },
    },
}
```

- `'version': 1` `1`

> `INFO` `DEBUG` `INFO`
> `WARNING` `ERROR` `CRITICAL`

---

12. gi. Web

`wsgi.py` Web Dja g

> WSGI Web Se e Ga e a l e face

> :
> `application`

```
"""
WSGI config for hozen project.

It exposes the WSGI callable as a module-level variable named ``application``.

For more information on this file, see
https://docs.djangoproject.com/en/5.0/howto/deployment/wsgi/
"""

# 環境変数操作のため
import os

# DjangoのWSGIアプリを取得する関数
from django.core.wsgi import get_wsgi_application

# 設定ファイルの場所を指定
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'hozen_another.settings')

# WSGIアプリ本体を作る（Gunicornが呼ぶ入口）
application = get_wsgi_application()
```

- `"""..."""` d c i g

- `import os`
- `from django.core.wsgi import get_wsgi_application`

- `os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'hozen_another.settings')`
  `'hozen_another.settings'`
  `settings.py` `setdefault`

- `application = get_wsgi_application()`
  `application`

> **`application`** `application`
> `gunicorn`
> `hozen_another.wsgi:application` **`hozen_another/wsgi.py`**
> **`application`**

> ⚠️ `wsgi.py`
> `hozen_another.settings`

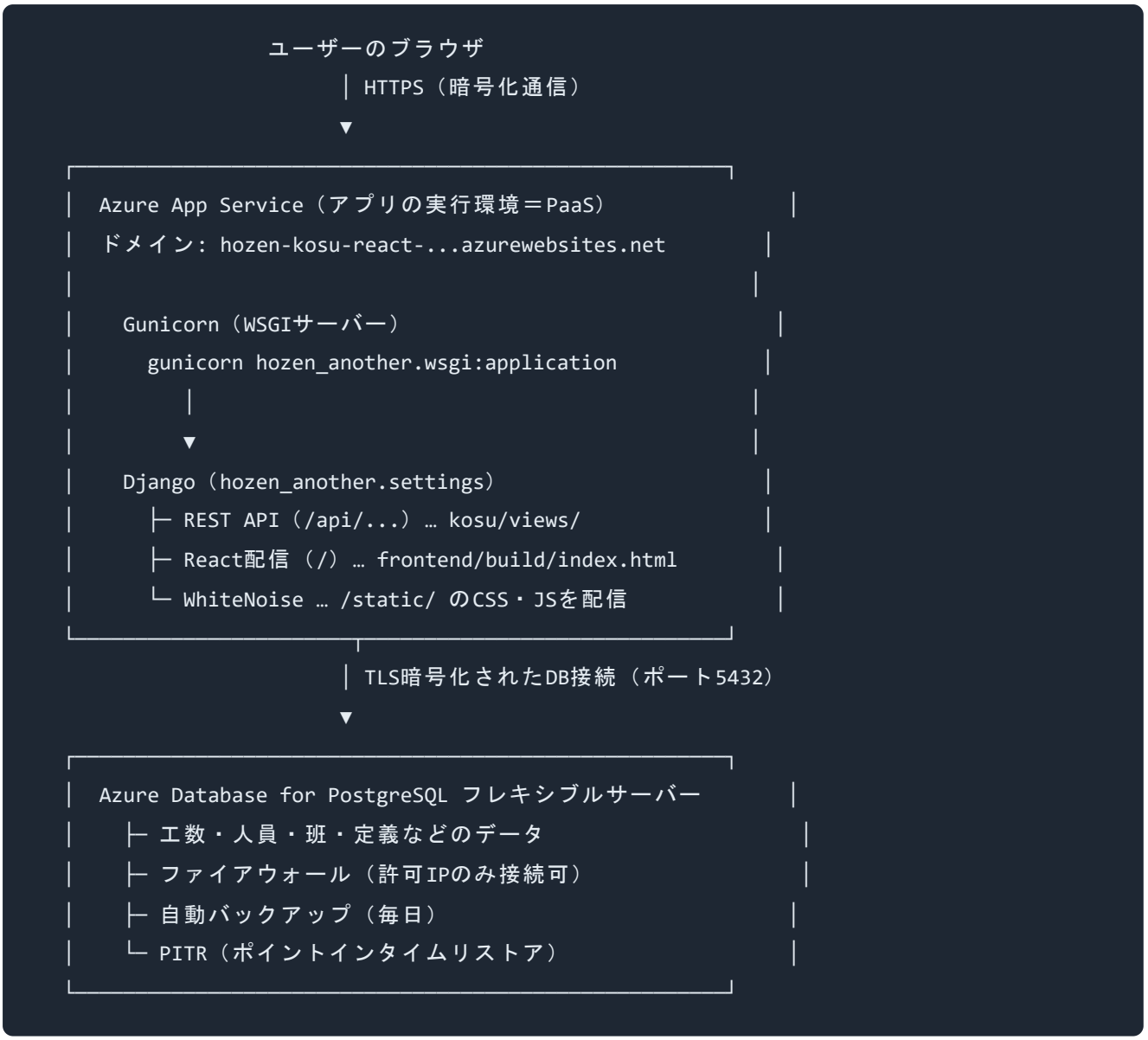
---

13. A e

Mic f A e

> cl d

13.1



13.2

A e A Se ice		
G ic		`wsgi.py` `application`
Whi eN i e		
A e Da aba e f P g eSQL		

PaaS Pla f a a Se ice

G ic G ee U ic

`runserver`

`gunicorn hozen_another.wsgi:application` `application`

13.3 A e Da aba e f P g eSQL

Fle ible Se e

	`DB_HOST` `xxx.postgres.database.azure.com`	
TLS		
PITR		

e all

DB IP

TLS T a La e Sec i

PITR P i -I -Ti e Re e

6 9

⚠ 2

A e PITR			

14.

14.1

:

```
① ローカルでテストを通す
   npm run test:run            # フロントのテストが全部緑か確認
   （バックエンドの動作も手元で確認）

② フロントエンドを本番ビルド
   cd frontend
   npm run build               # frontend/build/ に最新の成果物を作る

③ 静的ファイルを収集（プロジェクトルートで）
   python manage.py collectstatic   # staticfiles/ に集約（WhiteNoise配信用）

④ 変更をコミット・プッシュ
   git add -A
   git commit -m "（変更内容）"
   git push                    # GitHubへ

⑤ Azureへデプロイ
   （GitHub連携の自動デプロイ、またはAzure CLIで反映）

⑥ 本番でマイグレーションを実行（必要な場合のみ）
   python manage.py migrate    # 本番DBにモデル変更を反映
```


- `test:run`
- `npm run build`
- c llec a ic
- `frontend/build/` `staticfiles/` `.css.gz`
-

-

> ⚠️
>
> `npm run build` `collectstatic`

14.2

> ⚠️ **5**
>
> **`migrate`** **`--plan`** `python manage.py migrate --plan`

> `member`
>
> `makemigrations`
>
> `migrate`
>
> `kosu/migrations/`

> **`migrate --plan`**
>
> `--plan`

14.3

□ トップページが表示されるか（500/502エラーが出ないか）
□ ログインできるか
□ 主要機能（工数入力・一覧表示）が動くか
□ 今回変更した機能が意図どおり動くか
□ web_console.log にエラーが出ていないか（§11.10）

⚠️



15. HTTP

HTTP

HTTP

2		
3		
4		
5		

200			

202			`check_task_status`
301/302			`/login`
400			`validate_dates` `ALLOWED_HOSTS`
401			`/login`
403			`CSRF_TRUSTED_ORIGINS`
404			
405			`backup`
500			
502			
503			

> ⚠️ 400 500

> :
>
> - `CSRF_TRUSTED_ORIGINS`
> - `ALLOWED_HOSTS`
> - `wsgi.py`
> - `web_console.log`

16.

16.1

> :

```
① 症状を正確に把握する
    ・誰が、いつ、何をしたら、どうなったか
    ・エラーメッセージ・ステータスコード（§15）を聞く／見る
    ・スクリーンショットをもらう

② 範囲を切り分ける
    ・全員で起きる？ 特定ユーザーだけ？
    ・全機能ダメ？ 特定機能だけ？
    ・本番だけ？ ローカルでも再現する？
        → 全員・全機能ダメ → サーバー/DBレベルの障害（5xx系を疑う）
        → 特定機能だけ     → その機能のコード/データを疑う

③ ログを確認する
    ・web_console.log（§11.10）を見る
    ・Azureのアプリログ・Gunicornログを見る
    ・ブラウザのF12（開発者ツール）でステータスコードと通信内容を確認

④ 原因を特定する
    ・ステータスコード（§15）で4xx（送り手）か5xx（サーバー）か切り分け
    ・最近のデプロイ・設定変更が引き金になっていないか確認

⑤ 対処する
    ・軽微 → 設定修正・データ修正
    ・深刻 → 直前の正常な版へ戻す（ロールバック）
    ・データ破損 → バックアップ/PITR（§13）から復元

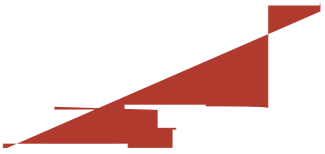
⑥ 記録する
    ・何が起きて、何をして直したかをメモに残す（次回のため）
```

16.2

		`npm run build`
		`CSRF_TRUSTED_ORIGINS`
	`ALLOWED_HOSTS`	
		`AsyncTask` `web_console.log`
		`USE_TZ`
	`DB_HOST`	

> ⚠️
>
> `git log`

16.3



・コードの問題 → 直前の正常なコミットへ戻して再デプロイ
・DBの問題 → Azure PITR（§13.3）で正常な時点へ復元

>

---

17. PDF

17.1



	`web_console.log`	

> ⚠️ `History`
> `AsyncTask` `delete_History_data`
> `delete_AsyncTask_data`
> `generate_*_backup`

17.2 PDF

`docs/` `.md` `docs/tools/build-pdf.js`

> : `docs/` `.md`
> `pdf.css` `docs/pdf/`

```
// docs配下の全 .md ファイルを docs/pdf/ にPDF変換するスクリプト
// 使い方:  node docs/tools/build-pdf.js
// ファイル操作モジュール
const fs = require("fs");
// パス操作モジュール
const path = require("path");
// Markdown→PDF変換ライブラリ
const { mdToPdf } = require("md-to-pdf");

// docs/ フォルダのパス
const docsDir = path.resolve(__dirname, "..");
// 出力先 docs/pdf/
const outDir = path.join(docsDir, "pdf");
// 適用するスタイルファイル
const cssPath = path.join(__dirname, "pdf.css");

// 出力先が無ければ作る
if (!fs.existsSync(outDir)) fs.mkdirSync(outDir, { recursive: true });

// docs直下の .md を対象（tools/pdf配下は除外）
const mdFiles = fs
  // docs/ の中身を一覧
  .readdirSync(docsDir)
  // .md だけ残す
  .filter((f) => f.endsWith(".md"))
  // 名前順に並べる
  .sort();
```

- `const fs = require("fs");` `require`
- `const { mdToPdf } = require("md-to-pdf");`
- `const docsDir = path.resolve(__dirname, "..");` `__dirname` `docs/tools/` `docs/`
- `const outDir = path.join(docsDir, "pdf");` `docs/pdf/`
- `const cssPath = path.join(__dirname, "pdf.css");`
- `if (!fs.existsSync(outDir)) fs.mkdirSync(...)` `recursive: true`
- `fs.readdirSync(docsDir).filter(...).sort()` `docs/` `.md`

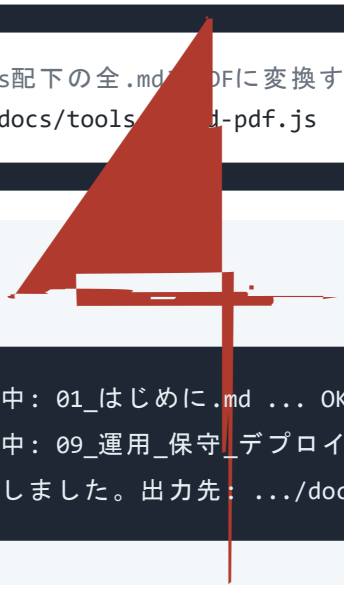
```
// 即時実行の非同期関数
(async () => {
  // 各 .md ファイルについて
  for (const file of mdFiles) {
    // 入力パス
    const src = path.join(docsDir, file);
    // 出力パス（拡張子を.pdfに）
    const dest = path.join(outDir, file.replace(/\.md$/, ".pdf"));
    // 進捗表示
    process.stdout.write(`変換中: ${file} ... `);
    // Markdown→PDF変換を実行（完了を待つ）
    const pdf = await mdToPdf(
      // 入力ファイル
      { path: src },
      {
        // スタイル適用
        stylesheet: [cssPath],
        pdf_options: {
          // A4サイズ
          format: "A4",
          // 余白
          margin: { top: "18mm", bottom: "18mm", left: "16mm", right: "16mm" },
          // 背景色も印刷
          printBackground: true,
        },
        // ブラウザ起動オプション
        launch_options: { args: ["--no-sandbox"] },
      }
    );
    // 変換に成功したら
    if (pdf && pdf.content) {
      // PDFファイルを書き出す
      fs.writeFileSync(dest, pdf.content);
      // 成功ログ
      console.log("OK ->", path.relative(docsDir, dest));
    } else {
      // 失敗ログ
      console.log("失敗");
    }
  }
  // 全完了
  console.log("\n完了しました。出力先:", outDir);
  // 正常終了
  process.exit(0);
// どこかでエラーが出たら
})().catch((e) => {
  // エラー表示
  console.error("エラー:", e);
```

```
  // 異常終了
  process.exit(1);
});
```

- `(async () => { ... })()` `async` `await`
- `for (const file of mdFiles)` `.md`
- `const dest = path.join(outDir, file.replace(/\.md$/, ".pdf"))` `file.replace(/\.md$/, ".pdf")` `.md` `.pdf` `/\.md$/` `.md`
- `const pdf = await mdToPdf({ path: src }, {...})` `await`
- `launch_options: { args: ["--no-sandbox"] }`
- `if (pdf && pdf.content) fs.writeFileSync(dest, pdf.content)`
- `.catch((e) => { ... process.exit(1); })`

: `docs/` `docs/pdf/`

```
# docs配下の全.md OFに変換する
node docs/tools d-pdf.js
```



```
変換中: 01_はじめに.md ... OK -> pdf/01_はじめに.pdf
変換中: 09_運用_保守_デプロイ.md ... OK -> pdf/09_運用_保守_デプロイ.pdf
完了しました。出力先: .../docs/pdf
```

> ⚠️ `md-to-pdf` `npm install md-to-pdf`

18.

`runserver` `port already in use`		
	`AsyncTask`	`web_console.log` `AsyncTask result`
		`validate_dates`
	`npm run build` `collectstatic`	
		`wsgi.py`
		`CSRF_TRUSTED_ORIGINS`
	`ALLOWED_HOSTS`	
		`web_console.log`
	`DB_HOST`	`.env`

		`USE_TZ` `astimezone`
`module not found`	`md-to-pdf`	`npm install md-to-pdf`

19.

1. `backup` `check_task_status` `download_file`
2. `generate_kosu_backup` `str(item.name)` `name` `str()`
3.
4. `validate_dates`
5. `download_file`
6. `settings.py`
7. `RotatingFileHandler` `maxBytes` `backupCount`
8. `wsgi.py` `application`
9.

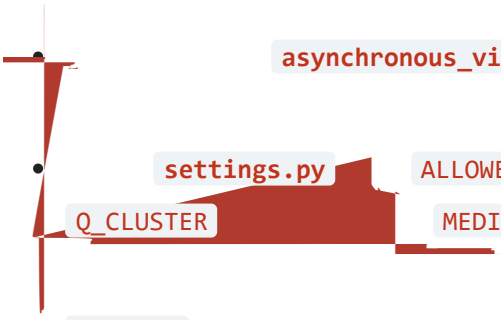
10.

11.

12. `Q_CLUSTER` `threading` `qcluster`

20.

- 3 `.env`
- `runserver` `migrate` `makemigrations` `qcluster` `collectstatic` `npm build`
- **`tasks.py`**



- **`asynchronous_views.py`**
- **`settings.py`** `ALLOWED_HOSTS` `CSRF_TRUSTED_ORIGINS` `DATABASES` `Q_CLUSTER` `MEDIA_ROOT` `LOGGING`
- **`wsgi.py`** `application`
-
-
- HTTP

-
- PDF `node docs/tools/build-pdf.js`

: 10_ . d